



Implementación de Cluster MySQL con Replicación GTID y Balanceo de Carga mediante HAProxy

Autores: Jorge Andrés Medina Urrutia · Santiago Vivas Amaya · Esteban Nicolás Sánchez Tobar · Alejandro Muñoz Franco · Juan José Solarte Pabón

Departamento de Ingeniería de Software

Alta disponibilidad · Replicación GTID · Balanceo de carga con HAProxy

Tabla de Contenido

01

Introducción

Motivación y tecnologías clave

02

Contexto / Problema

Problemática y requerimientos

03

Alternativas de Solución

Evaluación comparativa de arquitecturas

04

Diseño de la Solución

Arquitectura en tres capas

05

Implementación

Entorno técnico y fases

06

Pruebas

Resultados con Sysbench

07

Discusión de las Pruebas

Cumplimiento, lecciones y limitaciones

08

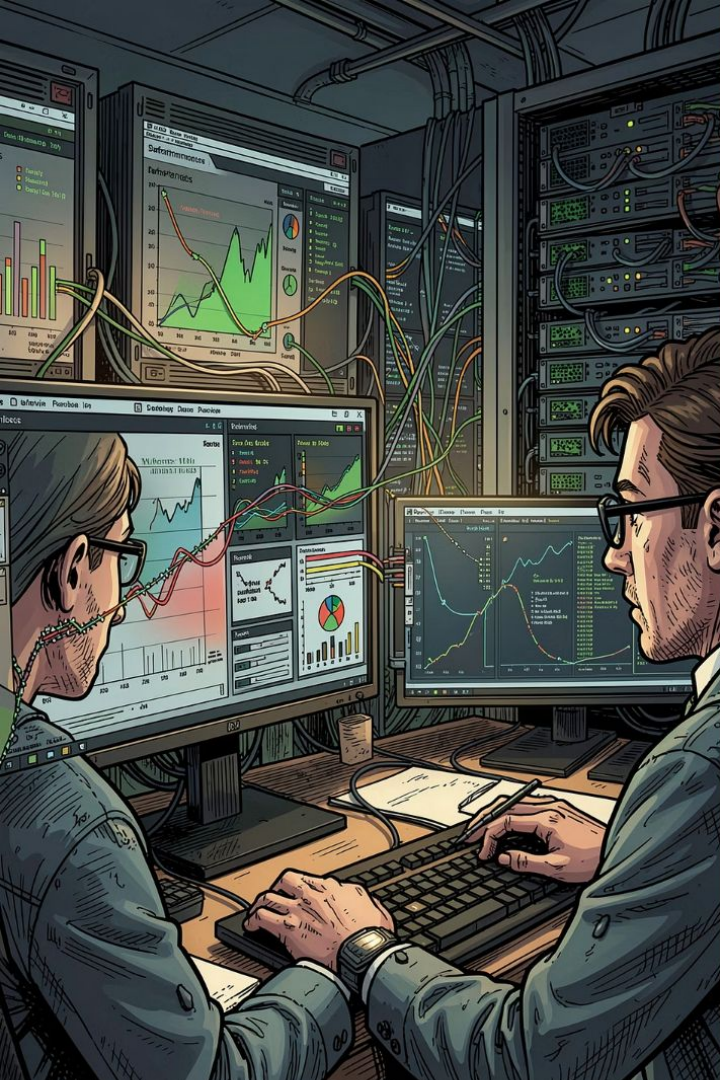
Conclusiones

Hallazgos y recomendaciones

09

Referencias

Bibliografía y repositorio



Introducción

Las aplicaciones empresariales modernas exigen **disponibilidad continua**, **rendimiento consistente** y **escalabilidad horizontal**. Los sistemas monolíticos de bases de datos no satisfacen estos requisitos en entornos de alta concurrencia.

MySQL

SGBD relacional open-source con replicación nativa y ecosistema maduro

Replicación GTID

Identificadores globales de transacción para consistencia y recuperación automatizada

HAProxy

Balanceador de carga TCP/HTTP de alta disponibilidad con health checks inteligentes

Contexto y Problema

Problemática identificada

- Escalabilidad limitada por un único servidor de base de datos
- Punto único de fallo sin mecanismo de conmutación
- Competencia entre operaciones de lectura y escritura
- Complejidad operacional en replicación tradicional
- Falta de visibilidad sobre el estado real de la replicación

Requerimientos del sistema

- Replicación maestro-esclavo con **dos nodos esclavos**
- Uso de **GTID** para auto-positioning automático
- Separación de tráfico: escritura (3307) / lectura (3308)
- Health checks inteligentes en puerto 9200
- Dashboard de monitoreo en puerto 8080
- Lag de replicación cercano a **cero**

Alternativas de Solución

Evaluación comparativa de tres arquitecturas candidatas según criterios ponderados.

Criterio	MySQL Group Rep. + ProxySQL	MySQL GTID + HAProxy 	Galera Cluster + HAProxy
Simplicidad	6	9	5
Disponibilidad	9	8	9
Rendimiento	8	9	7
Madurez	7	9	8
Coste operacional	6	9	5
Escalabilidad	8	9	7
Puntuación media	7.33	8.75 / 10	6.83

 **Solución seleccionada:** MySQL Replicación GTID + HAProxy — mayor simplicidad, madurez, bajo overhead y facilidad operativa.

Diseño de la Solución



Arquitectura en tres capas

- **Capa de acceso:** HAProxy como único punto de entrada
- **Capa de datos:** Cluster MySQL (1 maestro + 2 esclavos) con GTID
- **Capa de monitoreo:** Health checks HTTP en puerto 9200 y dashboard en 8080

Despliegue completo mediante **Docker Compose** sobre Vagrant VM.



Implementación



Entorno técnico

- Ubuntu 22.04 LTS · Vagrant VM
- MySQL 8.0.46 · HAProxy 2.8-alpine
- Docker · Docker Compose · Socat · Bash



Fases de despliegue

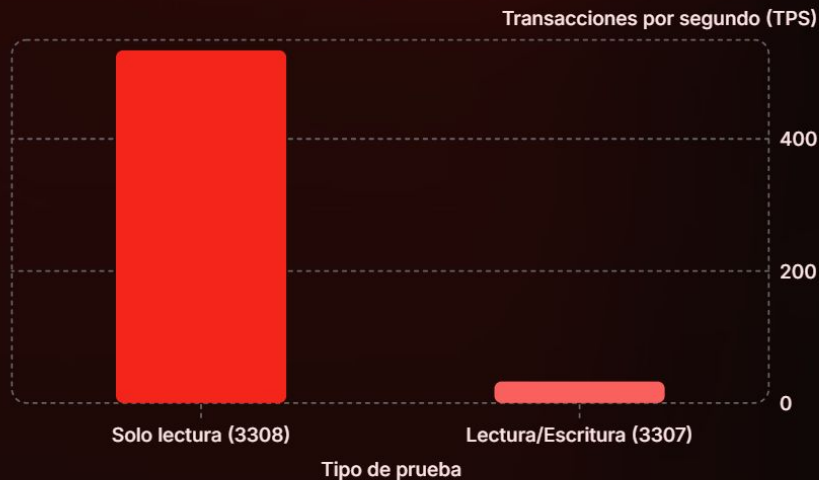
1. Creación del cluster MySQL con GTID
2. Integración de HAProxy
3. Validación y ajustes operativos



Problemas resueltos

- Redirección stdout/stderr en `healthcheck.sh`
- `bind=0.0.0.0` para exponer health checks
- Corrección de puerto interno 9200 en `HAProxy`
- Salto de línea final en `haproxy.cfg`
- `mysql_native_password` para Sysbench

Pruebas y Resultados



Métricas clave de rendimiento

- **Read-only (3308):** 32 077 transacciones · **534,09 TPS**
- **Read/Write (3307):** 1 954 transacciones · **32,51 TPS**
- Distribución de carga **50/50** entre esclavos
- Lag de replicación: **0 segundos** bajo carga normal
- Detección de fallo: **6,2 s** · Recuperación: **4,1 s**
- Overhead de HAProxy: **menor al 1%**

Discusión de las Pruebas

Requerimientos cumplidos

- Replicación GTID operativa con auto-positioning
- Separación efectiva de tráfico lectura/escritura
- Health checks funcionales y dashboard útil
- Distribución de carga dentro del margen esperado

Lecciones aprendidas

- Importancia de redes Docker y bind addresses
- Diferencia entre `stdout` y `stderr` en Bash
- Ventaja operativa del GTID auto-positioning

Limitaciones identificadas

- Failover del maestro **no automatizado**
- Seguridad insuficiente para entornos de producción
- Falta monitoreo avanzado (Prometheus/Grafana)
- Sin estrategia formal de backup y recovery

Conclusiones y Referencias

Solución validada

Cumple todos los requerimientos funcionales y no funcionales con rendimiento demostrado

Mejor alternativa

MySQL GTID + HAProxy: mayor simplicidad, madurez y bajo overhead (8,75/10)

Alta disponibilidad

Arquitectura reproducible, modular y escalable para lecturas en producción

Próximos pasos

Seguridad, monitoreo avanzado, backups y failover automático del maestro

Referencias

- Kleppmann, M. *Designing Data-Intensive Applications*. O'Reilly, 2017.
- Oracle. *MySQL 8.0 Reference Manual — Replication*.
- Baron Schwartz et al. *High Performance MySQL*. O'Reilly.
- HAProxy Technologies. *Documentación oficial de HAProxy*.
- Repositorio de evidencias: github.com/Nico4ll/LabHaproxySql